

NAME: _____

ID: _____

COMPSCI 1XC3: Computer Science Practice and Experience: Development Basics
PRACTICE EXAM

Instructions

- Unless otherwise specified, the filenames of the programs are `main.c`.
- If not explicitly written, you can assume `#include <stdio.h>` for any programs that we provide or that you write.
- Unless otherwise specified, be sure to write the complete output, paying attention to the formatters. For example, given:

```
1 printf("i = %d \n", 10);  
2 printf("j = %d ", 12);
```

Correct:

```
i = 10  
j = 12
```

Incorrect:

```
10 12
```

Section 1: Basics

Question 1 ☐ Examine the program. What will print?

```
1 #include <stdio.h>
2
3 int bar(){
4     auto int var;
5     printf("var = %d \t address = %p \n", var++, &var);
6
7     var = 0;
8     {
9         auto int var = 3;
10        printf("var = %d \t address = %p \n", var, &var);
11    }
12    var++;
13    printf("var = %d \t address = %p \n", var++, &var);
14
15    return var;
16 }
17
18 int main(){
19     int var = bar();
20     printf("var = %d \t address = %p \n", var++, &var);
21 }
```

Question 2 ☐ Examine the following two programs.

main_one.c:

```
1 int main() {
2     int a[5];
3     a = {1,2,3,4,5};
4
5 }
```

main_two.c:

```
1 int main() {
2     int a[5] = {1,2,3,4,5};
3     int b[5];
4     b = a;
5 }
```

1. What is the outcome of `gcc main_one.c`?
2. What is the outcome of `gcc main_two.c`?
3. Describe why these outcomes occur.

Question 3 [] Examine the program.

```
1 #include <stdio.h>
2
3 int main(int argc, char *argv[]) {
4     for (int i = 0; i < argc; i++) {
5         printf("argv[%d] = %s\n", i, argv[i]);
6     }
7 }
```

Part A: You compile and run the program with the command:

```
gcc main.c -o main.exe && ./main.exe
```

What is the program output?

Part B: You compile and run the program with the command:

```
gcc main.c && ./a.out hello
```

What is the program output?

Question 4 [] Examine the program.

```
1 #include <stdio.h>
2 #define PRINT_INT(n) printf(#n " = %d\n", n)
3 #define SCALE(x) (x * 10)
4
5 int main() {
6     PRINT_INT(SCALE(2+1));
7     #undef SCALE
8     #define SCALE(x) ((x) * 10)
9     PRINT_INT(SCALE(2+1));
10 }
```

What does it output?

Question 5 [] Examine the program.

```
1 #include <stdio.h>
2
3 char foo(){
4     static int var;
5     return var++;
6 }
7
8 int bar(){
9     static int var = 0;
10    var++;
11    return var;
12 }
13
14 int main(){
15     for (int i = 0; i < 3; i++) foo();
16     for (int i = 0; i < 3; i++) bar();
17     printf("foo var = %d\n", foo());
18     printf("bar var = %d\n", bar());
19 }
```

1. What will the program print?
2. We want to debug the program with gdb. Write the command to compile the program with debug information to an executable called `main.out`.
3. You start gdb and want to observe the value of `foo`'s local `var` in each call. Write the gdb command to set a breakpoint on the `foo` function.

Section 2: Compilation

Question 6 [] Examine the code. Write the command(s) that will produce the following output:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(int argc, char* argv[]) {
5     printf("The weather today is ");
6     #ifdef WEATHER
7         printf("%s.\n", WEATHER);
8     #else
9         printf("unknown.\n");
10    #endif
11    printf("The temperature today is ");
12    #ifdef TEMP
13        printf("%d.\n", TEMP);
14    #else
15        printf("unknown.\n");
16    #endif
17 }
```

Expected output:

```
The weather today is rainy.
The temperature today is 22.
```

Question 7 [] You are given `math_utils.h`, `geometry.h`, and `main.c` as shown.

```
1 // math_utils.h
2 typedef struct {
3     int x;
4     int y;
5 } Point;
6 int add(int a, int b);
```

```
1 // geometry.h
2 #include "math_utils.h"
3 int distance(Point a,
4              Point b);
```

```
1 // main.c
2 #include "math_utils.h"
3 #include "geometry.h"
4 int main() {
5     Point p1 = {0, 0};
6     Point p2 = {3, 4};
7     return 0;
8 }
```

When compiling this program, you get an error:

```
error: conflicting types for 'Point'
```

Modify `math_utils.h` to prevent this error.

Question 8 [] You have the following files in your directory:

```
main.c
utils.c
utils.h
```

`main.c` includes `utils.h`. `utils.h` defines a macro `MAX 100`.

Part A: Write the `gcc` command to run *only the preprocessor* on `main.c` and save the output to `main.i`. What differences would you expect to see in `main.i` versus `main.c`?

Part B: Write the `gcc` command to *compile but not link* each of `main.c` and `utils.c`. What files does this produce?

Part C: Write the `gcc` command to *link* the files produced in Part B into an executable called `program`.

Part D: Write a *single* `gcc` command that compiles and links everything into `program` in one step. When would you prefer this over the approach in Parts B and C?

Part E: Explain why the following command is not the intended use:

```
gcc main.c utils.c utils.h -o program
```

Part F: You want to debug `program` using `gdb`. Write the `gcc` commands to compile `main.c` and `utils.c` to object files and link them into an executable called `program` with full debugging support. What extra flag did you add, and at which steps must it be passed?

Part G: Write a complete Makefile for this three-file program. It should include the following rules:

- **all:** compiles `main.c` and `utils.c` to object files and links them into `program`
- **main.o:** compiles `main.c` to an object file
- **utils.o:** compiles `utils.c` to an object file
- **debug:** builds `program` with debug symbols enabled
- **clean:** removes all `.o` files and `program`

Section 3: Memory Model & Pointers

Question 9 ☐ Examine the program.

```
1 #include <stdio.h>
2
3 void f(int i, int *p) {
4     i = 5;
5     *p = 10;
6     printf("Inside f: i = %d\n", i);
7 }
8
9 int main() {
10     int x = 1;
11     f(x, &x);
12     printf("In main: x = %d\n", x);
13     return 0;
14 }
```

What will print?

Question 10 ☐ What is the composition of array **a** after these lines execute?

```
1 int a[10], *p, *q, i;
2
3 p = &a[8];
4 q = p - 3;
5 p -= 6;
6
7 *q = 202;
8 *p = 1;
```

Index	0	1	2	3	4	5	6	7	8	9
Value										

Question 11 ☐ Examine the code.

```
1 #include <stdlib.h>
2
3 typedef struct {
4     struct { int x; int y; } pos;
5     int     lives;
6     double  invincible_until;
7 } Player;
8
9 void reset_player(Player *p);
10
11 Player spawn = {{0, 0}, 3, 0.0};
12 Player *roster;
13 void (*default_reset)(Player *) = reset_player;
14
15 void reset_player(Player *p) {
16     static int reset_count = 0;
17     Player tmp = *p;
18     tmp.lives = 3;
19     reset_count++;
20 }
21
22 int main() {
23     const char *tag = "p1";
24     Player *party = (Player *)malloc(4*sizeof(Player));
25     roster = party;
26     default_reset(&party[0]);
27     free(party); party = NULL;
28 }
```

Part A: What are the types of `roster`, `default_reset`, `reset_player`, `tag`, and `party`?

Part B: Where are the following variables and functions located based on the C memory model? Regions: Stack, Heap, Global/static (`.data` and `.bss`), Code (`.text`).

Part C: `tmp` is initialized with `*p`, where `p` points to a dynamically-allocated `Player`. Does this mean `tmp` is on the heap? Explain why or why not.

Question 12 ☐ Examine the program.

```
1 #include <stdio.h>
2
3 int main() {
4     short arr[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
5
6     short *p = arr;
7     long *ip = (long *)p;
8     ip += 1;
9     p += 1;
10
11     printf("%hd\n", *p);
12     printf("%hd\n", *(short *)ip);
13 }
```

What will print?

Question 13 ☐ Examine the code.

```
1 #include <stdio.h>
2
3 void print_bits(unsigned char b) {
4     for (int i = 7; i >= 0; i--) {
5         printf("%d", (b >> i) & 1);
6     }
7     printf("\n");
8 }
9
10 int main() {
11     unsigned short x = 2;
12     unsigned char *p = (unsigned char *)&x;
13     printf("Address of x: %p\n", &x);
14     printf("Byte 0:\n");
15     print_bits(p[0]);
16     printf("Byte 1:\n");
17     print_bits(p[1]);
18 }
```

What will print? The first line of output is provided. The first line is provided:

Address of x: 0x7ffdf861bfde

Question 14 [] Given the following definitions:

```
1 typedef struct {  
2     int i;  
3     double d;  
4 } sNumber;
```

```
1 typedef union {  
2     int i;  
3     double d;  
4 } uNumber;
```

Part A: Declare and initialize an `sNumber` variable called `s_value` with `i = 5` and `d = 3.14`. Then declare and initialize a `uNumber` variable called `u_value` with `i = 5`.

Part B: What does `sizeof(s_value)` return? What does `sizeof(u_value)` return? Explain why they differ. Assume `sizeof(int) = 4` and `sizeof(double) = 8`.

Part C: After the following code runs, what is the value of `u_value.i`? Explain what happened.

```
1 u_value.i = 5;  
2 u_value.d = 3.14;  
3 printf("%d\n", u_value.i);
```

Part D: Would the same thing occur with `s_value`? Why or why not?

Section 4: Dynamic Memory

Question 15 [] Fill in each missing segment of code with a call to the most appropriate dynamic memory allocation function to make the code produce the expected output.

Part A: Allocate an array of 5 shorts. Expected output: 0 0 0 0 0

```
1 short *scores = ; /* TODO: complete the line */
2 for (int i = 0; i < 5; i++) {
3     printf("%hd ", scores[i]);
4 }
5 printf("\n");
6 // Expected output: 0 0 0 0 0
7 free(scores);
```

Part B: Allocate a large log buffer whose size is determined at runtime by a long int. Initialization does not matter since every byte will be written before being read. Expected output: OK

```
1 long buf_size = 5000000L;
2 char *log_buf = ; /* TODO: complete the line */
3 log_buf[0] = 'O';
4 log_buf[1] = 'K';
5 log_buf[2] = '\0';
6 printf("%s\n", log_buf);
7 // Expected output: OK
8 free(log_buf);
```

Part C: Grow an existing array of 3 ints to hold 6 without losing the original values. Expected output: 10 20 30 40 50 60

```
1 int *arr = malloc(3 * sizeof(int));
2 for (int i = 0; i < 3; i++){
3     arr[i] = (i + 1) * 10;
4 }
5 arr = ; /* TODO: complete the line */
6 for (int i = 3; i < 6; i++) {
7     arr[i] = (i + 1) * 10;
8 }
9 for (int i = 0; i < 6; i++) {
10    printf("%d ", arr[i]);
11 }
12 printf("\n");
13 // Expected output: 10 20 30 40 50 60
14 free(arr);
```

Question 16 [] Given the following for Parts A–D:

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <stdlib.h>
4
5 typedef struct {
6     char *name;
7     int score;
8 } Player;
```

Part A: Write a function `createPlayer(char *name, int score)` that dynamically allocates a `Player` and returns a pointer to it (the caller owns the memory). How many bytes do you need to allocate for the `name` field?

Part B: Write a function `comparePlayers(Player *a, Player *b)` that returns a negative value if `a`'s name comes before `b`'s alphabetically, 0 if equal, and positive if after. Use the appropriate function from `string.h`.

Part C: Write a function `fullTitle(Player *p, char *title)` that returns a newly heap-allocated string combining the title and name, e.g., `fullTitle(p, "Queen") → "Queen Alice"`. How does the length of the result relate to the lengths of the inputs?

Part D: Write a function `freePlayer(Player *p)` that frees all heap memory associated with a player. What order must the `free` calls happen in, and why?

Question 17 [] Overview for Parts A–E: A course has 3 graders. When exams come in, you need a rule to assign each exam to a grader based on the student ID so that the same student always goes to the same grader. At the top of the program, the number of graders is defined as a macro: `#define NUM_GRADERS 3`. Your assignment rule should continue to work if `NUM_GRADERS` is changed.

Part A: Write code to define the following:

1. An `Exam` struct with two fields: a student ID (`int`) and a grade (`float`).
2. A `Node` struct for a singly linked list where each node holds an `Exam` and a pointer to the next node.
3. An array of 3 `Node` pointers called `graders`, initialized to `NULL`. For the remaining questions, assume `graders` is a global variable.

Part B: Implement a function `insert` that takes a pointer to an `Exam` and assigns it to the correct grader by prepending it to that grader's list. `insert` will only be called with a valid `Exam` initialized with a student ID and a grade of `-1.0`.

Part C: Implement a function `enter_mark` that takes a student ID (`int student_id`) and a grade (`float grade`). If the student's exam is already in the system, update its grade. If not, create a new `Exam` with the given student ID and grade and insert it.

Part D: Implement a function `get_grade` that takes a student ID and returns a pointer to the student's `Exam`, or `NULL` if no exam with that student ID is found. Note that a grade of `-1.0` means the exam has not yet been graded.

Part E: Implement a function `cleanup` that frees all nodes in all grader lists.